CEFET-MG	CEFET – MG / CAMPUS LEOPOLDINA CURSO: ENGENHARIA DE CONTROLE E AUTOMAÇÃO AVALIAÇÃO PRÁTICA DE *ESTRUTURA DE DADOS* NOME:_____ DATA: 26/04/2016 VALOR: 30 pontos NOTA:_____ TEMPO DE DURAÇÃO: 1h 30min TEMPO MÍNIMO: 30 min

Questão 1: Considerando o código do ANEXO I desenvolva os seguintes procedimentos:

a) **void** RetiraTodos(TipoLista *Lista , TipoItem Item); que retira da lista todas as ocorrências do item passado como parâmetro. **(6 pontos)**

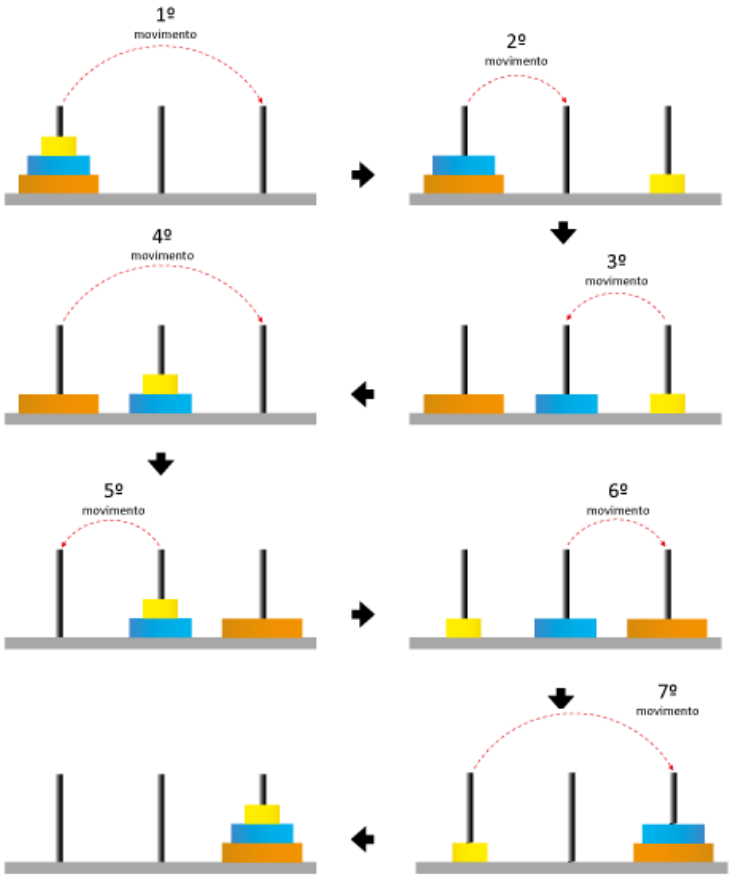
b) **void** InsereOrdenado(TipoItem x , TipoLista * Lista); que insere em uma lista ordenada em ordem crescente um novo item na sua posição. Considere nesse caso que todos os elementos são inseridos utilizando este procedimento. **(6 pontos)**

Questão 2: Considerando o código do ANEXO II:

a) Um dos problemas da implementação de filas por arranjo é que mesmo com uma das posições do vetor ainda vazia, o código informa que a fila está cheia. Modifique as estruturas e procedimentos para que isso não aconteça. **(6 pontos)**

b) Crie o procedimento **void** Imprime (TipoFila *Fila); que imprime os elementos da fila. **(6 pontos)**

Questão 3 (6 pontos): O brinquedo conhecido como torre de hanoi é um "quebra-cabeça" que consiste em uma base contendo três pinos, em um dos quais são dispostos alguns discos uns sobre os outros, em ordem crescente de diâmetro, de cima para baixo. O problema consiste em passar todos os discos de um pino para outro qualquer, usando um dos pinos como auxiliar, de maneira que um disco maior nunca fique em cima de outro menor em nenhuma situação. Observe a ilustração:



Considerando o código do ANEXO III, crie um programa declarando todas as variáveis e chamando todos os procedimentos e funções necessários para resolver uma torre com 3 peças. Imprima o estado da pilha a cada movimentação das peças.

```
ANEXO I - Lista apontador

typedef int TipoChave;
typedef struct {
        TipoChave Chave;
        /* outros componentes */
} TipoItem;
typedef struct TipoCelula *TipoApontador;
typedef struct TipoCelula {
        TipoItem Item; TipoApontador Prox;
} TipoCelula;

typedef struct {
        TipoApontador Primeiro , Ultimo;
} TipoLista ;

void FLVazia( TipoLista * Lista )
{       Lista->Primeiro = (TipoApontador) malloc( sizeof ( TipoCelula ) ) ;
        Lista->Ultimo = Lista->Primeiro ; Lista->Primeiro->Prox = NULL;
}

int Vazia( TipoLista Lista )
{ return ( Lista . Primeiro == Lista . Ultimo ) ; }

void Insere(TipoItem x ,     TipoLista * Lista )
{       Lista->Ultimo->Prox = (TipoApontador) malloc( sizeof ( TipoCelula ) ) ;
        Lista->Ultimo = Lista->Ultimo->Prox ;  Lista->Ultimo->Item = x;
        Lista->Ultimo->Prox = NULL;
}

void Retira(TipoApontador p,        TipoLista *Lista , TipoItem *Item)
{       /*--O item a ser retirado e o seguinte ao apontado por p--*/
        TipoApontador q;
        if ( Vazia(*Lista )     ||      p == NULL    ||      p->Prox == NULL){
                printf ( " Erro : Lista vazia ou posicao nao existe \n" ) ;
                return ;
        }
        q = p->Prox; *Item = q->Item ;  p->Prox = q->Prox ;
        if ( p->Prox == NULL ) Lista->Ultimo = p; free (q);
}
```

```
ANEXO II - Fila arranjo

#define MAXTAM 1000
typedef int TipoApontador;
typedef int TipoChave;
typedef struct {
        TipoChave Chave;
        /* outros componentes */
} TipoItem;

typedef struct {
        TipoItem Item[ MAXTAM ] ;
        TipoApontador Frente , Tras;
} TipoFila ;

void FFVazia( TipoFila * Fila )
{       Fila->Frente = 1; Fila->Tras = Fila->Frente ;
}

int Vazia( TipoFila Fila )
{ return ( Fila . Frente == Fila . Tras ) ; }

void Enfileira (TipoItem x , TipoFila * Fila )
{               if ( Fila->Tras % MAXTAM + 1 == Fila->Frente)
                        printf ( " Erro f i l a está cheia\n" ) ;
                else {
                        Fila->Item [ Fila->Tras - 1] = x;
                        Fila->Tras = Fila->Tras % MAXTAM + 1;
                }
}
void Desenfileira ( TipoFila * Fila , TipoItem *Item)
{       if ( Vazia(*Fila ) )
                printf ( "Erro f i l a esta vazia \n" ) ;
        else {
                *Item = Fila->Item [ Fila->Frente - 1];
                Fila->Frente = Fila->Frente % MAXTAM + 1;
        }
}
```

ANEXO III – Pilha arranjo

```
#define  MAXTAM 1000
typedef  int  TipoApontador;
typedef  int  TipoChave;
typedef  struct  {
      TipoChave Chave;
      /* --- outros  componentes--- */
}  TipoItem;
typedef  struct  {
      TipoItem  Item[ MAXTAM ] ;
      TipoApontador Topo;
}  TipoPilha ;

void  FPVazia( TipoPilha  *Pilha)
{  Pilha->Topo  =  0 ;  }

int  Vazia( TipoPilha  Pilha)
{  return  ( Pilha.Topo ==  0 );  }

void  Empilha(TipoItem  x ,  TipoPilha  *Pilha)
{     if ( Pilha->Topo == MaxTam)
            printf ( "Erro : pilha  esta  cheia\n" ) ;
      else  {
            Pilha->Topo++;
            Pilha->Item [ Pilha->Topo - 1] = x ;
      }
}

void  Desempilha( TipoPilha  *Pilha ,  TipoItem  *Item)
{     if( Vazia(*Pilha ) )
            printf ( "Erro :  pilha  esta  vazia \n" ) ;
      else  {
            *Item = Pilha->Item [ Pilha->Topo  - 1];
            Pilha->Topo --;
      }
}

int  Tamanho( TipoPilha  Pilha)
{  return  ( Pilha.Topo) ;  }
```